TutorBot

Contributors :

1) Jay Pandya (JXP230045)
2) Namatha Chintakunta (NXC230041)
3) Nanddanaa Bobba (NXB240002)

❖ Table of Contents

❖ Demo Video link

➢ **Demo video -**
➔ https://youtu.be/_XRRUv9eFmo

➢ **Flask appliation, Grammar module & Evaluation (Extended)**
➔ https://youtu.be/700_6TfZG7A

➢ **Mood Analysis & Evaluation (Extended)**
➔ https://youtu.be/p22mcM5mWzQ

➢ **Prediction module (Extended)**
➔ https://youtu.be/W2mwtCHQljY

❖ Github link :

➔ https://github.com/jpandya1161/Tutor-Bot

1. Project Description

This project is centered around building a smart, bilingual **TutorBot** — an AI-powered conversational assistant designed to help users improve their language skills in **English and French**. The application acts as an intelligent language tutor that can complete partially typed sentences, correct grammar (in French), detect the user's mood or sentiment, and carry out meaningful conversations in a multilingual context.

The core purpose is to make language learning more interactive, intuitive, and emotionally aware. Instead of offering static suggestions, the system provides **dynamic, context-aware feedback** through a conversational interface that feels like speaking to a real tutor.

The primary goals include:

- Enabling **bilingual chatbot interaction** powered by OpenAI's GPT and DeepL translation API.
- Offering **grammar correction** for French learners.
- Providing **sentence predictions** in French from incomplete phrases.
- Detecting the **user's sentiment and mood** to personalize the conversation.

The motivation behind this project stems from the need for intelligent, real-time language support tools that are both educational and emotionally responsive. By combining **NLP generation, classification, translation, and correction** in one web interface, this TutorBot showcases how advanced AI can enhance both learning outcomes and user engagement in multilingual education.

2. Project Explanation

📊 Base Application :

File: `app.py`

1. **Imports and Setup:**
   - Flask modules: Used to create the web server and handle requests/responses.
   - OpenAI and dotenv: For interacting with the GPT model and securely loading API keys.
   - requests: Used for calling external APIs (DeepL for translation).
   - Custom modules: mood.py, predict.py, and grammer.py provide specific NLP functionalities.
   - Loads environment variables from a .env file. Initializes the Flask app instance.

2. **API Keys and Clients:**
   - Retrieves keys from environment.
   - Sets up OpenAI and DeepL API usage.

3. **Language Utilities:**
   - detect_language(text)

   Uses the DeepL API to detect the source language of a given text.
   - translate_text(text, target_lang)

   Translates text to a given target_lang using the DeepL API.

4. **Routes & App runner:**
   - Gets input text from the frontend.
   - Detects the language of the input and translates it to the opposite language:

   French → English

   English → French
   - Calls the analyze_text() function from mood.py to detect the mood of the user's input.
   - Uses predict_sentences() from predict.py to suggest sentences based on partial input.

   Can return predictions in English and French.If error occurs, a fallback response is provided.
   - Detects if the user input is in French.

   If yes: Calls correct_french_sentence() to apply grammar correction.

📊 Grammar Module :

File: `grammer.py`

1. **Imports and Setup:**
   - Imports the SpellChecker class from the pyspellchecker library.
   - This tool uses a built-in dictionary to detect and correct misspelled words.

2. **Functionality:**
   - Defines a function correct_french_sentence that takes a single string argument sentence.
   - Splits the input sentence into **individual words** based on whitespace.
   - Applies spelling correction word-by-word using the spell checker.
   - spell.correction(word) returns the most likely correct version of the word.
   - If the word is already correct or not found in the dictionary, it uses the original word (or word fallback).
   - Joins the corrected words back into a full sentence.
   - If the corrected sentence is **different** from the original, return the corrected version.
   - Otherwise, return None to indicate **no correction was needed**.

3. **Limitations:**
   - The pyspellchecker library only corrects individual words, not grammar or sentence structure.
   - It may not understand context, leading to incorrect suggestions.
   - Limited to the internal dictionary of known words; can't fix grammar or typos in multi-word expressions.

📊 Grammar Evaluation :

File: `grammer_error_analysis.ipynb`

1. **Imports:**
   - re: for regex-based sentence splitting.
   - random: to introduce random errors.
   - pandas: for organizing and analyzing data.
   - SequenceMatcher: to compute string similarity between original and corrected sentences.

- correct_french_sentence: the correction function being tested (from your custom grammar module).

2. **Setup:**
   - This Python code is a testing and evaluation pipeline for a French spell-checking function (correct_french_sentence).
   - It simulates noisy input, corrects it, and measures how well the correction restores the original sentence.

3. **Functionality:**
   - This function splits a **French paragraph** into sentences using punctuation (., !, ?) followed by a space.
   - Reads the content of book.txt in UTF-8 encoding
   - Splits the text into sentences. Keeps only sentences with **more than 4 words** (to ensure substance). Selects the **first 3000** sentences for the experiment.
   - This function introduces **1–2 random errors** per sentence:
   - Either **reverses a word** (e.g., "bonjour" → "ruojnob")
   - Or **drops the first letter** (e.g., "bonjour" → "onjour")
   - For each corrupted sentence, apply your correct_french_sentence() function.
   - If no correction is made (None returned), use the original corrupted sentence as fallback.
   - This gives you corrected predictions to evaluate.
   - Compares the original clean sentence vs. corrected version to see how close the correction was to the true sentence.

File: predict.py

1. **Imports and Setup:**
   - `torch`: Used for tensor operations during model inference.
   - `transformers`:
     - `AutoModelForCausalLM`, `AutoTokenizer`: Load French GPT-2 model.
     - `MarianMTModel`, `MarianTokenizer`: Load MarianMT models for translation.
   - Pretrained Models Used:
     - `asi/gpt-fr-cased-small`: French GPT-2 for sentence generation.
     - `Helsinki-NLP/opus-mt-en-fr`: English to French translator.
     - `Helsinki-NLP/opus-mt-fr-en`: French to English translator.
   - Configures tokenizer padding and sets all models to evaluation mode.

2. **Translation Utilities:**
   - `translate_en_to_fr(texts)`: Translates English list to French using MarianMT with tokenization and decoding.

· `translate_fr_to_en(texts)`: Translates French list to English using MarianMT.

3. **French Text Generation:**
   · `generate_french_autocomplete(prompt_fr, max_new_tokens=5, num_predictions=5)`:
   · Generates continuations for a French prompt using GPT-2.
   · Uses top-k sampling (`k=40`), top-p (`p=0.95`), and temperature (`0.7`).
   · Filters duplicates, ensures completions start with the prompt, and truncates at the first period.
   · Returns 3 high-quality completions.

4. **Prediction Dispatcher:**
   · `predict_sentences(user_text)`:
   · Detects input language via character check.
   · If English: Translates to French → generates completions → translates back to English.
   · If French: Generates completions directly → translates to English.
   · Returns a dictionary: `{"french": [...], "english": [...]}`.

5. **Models & Techniques Used:**
   · MarianMT: Encoder-decoder models for translation.
   · GPT-2: Autoregressive decoder-only model for French sentence completion.
   · SentencePiece and BPE tokenization.
   · Decoding: Top-k, top-p, temperature scaling.
   · Zero-shot multilingual pipeline with no custom training.

📊 Mood Evaluation :

File: `cahtbot_mood_sentiment.py`

1. Imports and Setup:

- Imports required libraries:
  - `transformers.pipeline`, `AutoModelForSequenceClassification`, and `AutoTokenizer` for model loading and inference.
  - `pandas` and `LabelEncoder` for decoding predicted emotion labels.
- Loads:
  - **Sentiment model**: `distilbert-base-uncased-finetuned-sst-2-english`
  - **Trained GoEmotions model**: Loaded from a local zipped folder (`fine-tuned-goemotions`).

- Initializes Hugging Face pipelines for both sentiment and emotion detection.

2. Functionality:

- Defines `analyze_text(text)`:
  - Accepts a user sentence.
  - Predicts **sentiment** (positive/negative).
  - Predicts **mood/emotion** using the fine-tuned GoEmotions model.
  - Decodes the raw label (e.g., `LABEL_7`) to the actual emotion name using `LabelEncoder`.
  - Returns a formatted string with both sentiment and mood.
- Includes error handling to avoid runtime failures for invalid inputs or model errors.

3. Limitations:

- The model handles **single-label predictions** only (top-1 emotion per sentence).
- No handling of **multi-sentence context** or **emotion overlaps**.
- Performance is dependent on training data quality and label balance.

📊 Mood Evaluation & Error Analysis Module

File: `mood_error_analysis.ipynb`

1. Imports and Setup:

- Uses `os`, `zipfile`, `pandas`, `matplotlib`, and `seaborn` for data handling and visualization.
- Loads:
  - The **fine-tuned mood model** and tokenizer from zip folder.
  - A cleaned evaluation dataset: `goemotions_300_cleaned.csv`
- Initializes Hugging Face classifier pipeline and fits `LabelEncoder` for decoding.

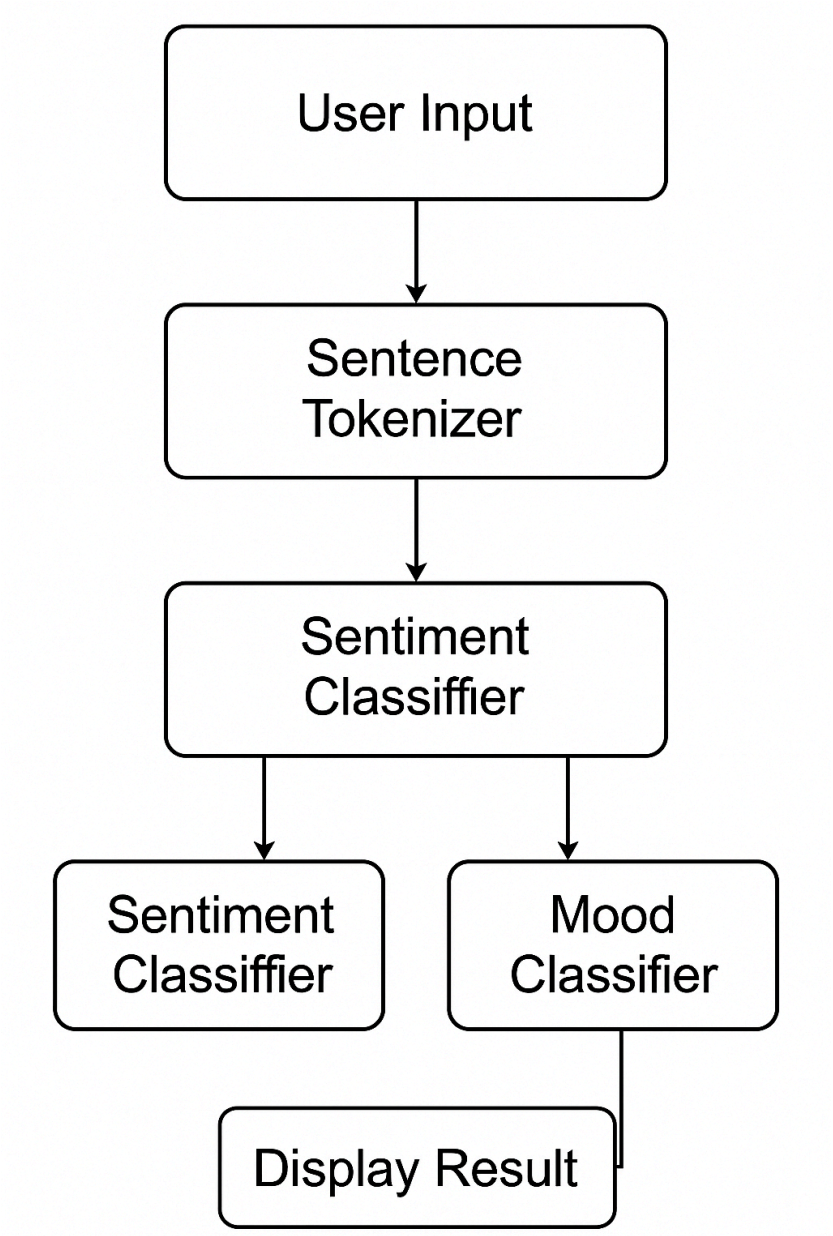
2. Functionality:

- Runs the **trained mood model** on 300 labeled test sentences.
- Decodes predicted labels and compares them to actual emotion labels.
- Computes and displays:
  - **Accuracy** (~80%)
  - **Classification report**: precision, recall, F1-score using `sklearn`
  - **Confusion matrix** heatmap for visual inspection of misclassifications
- Extracts and prints **error samples** (text + actual vs predicted emotion) for manual analysis.

3. Limitations:

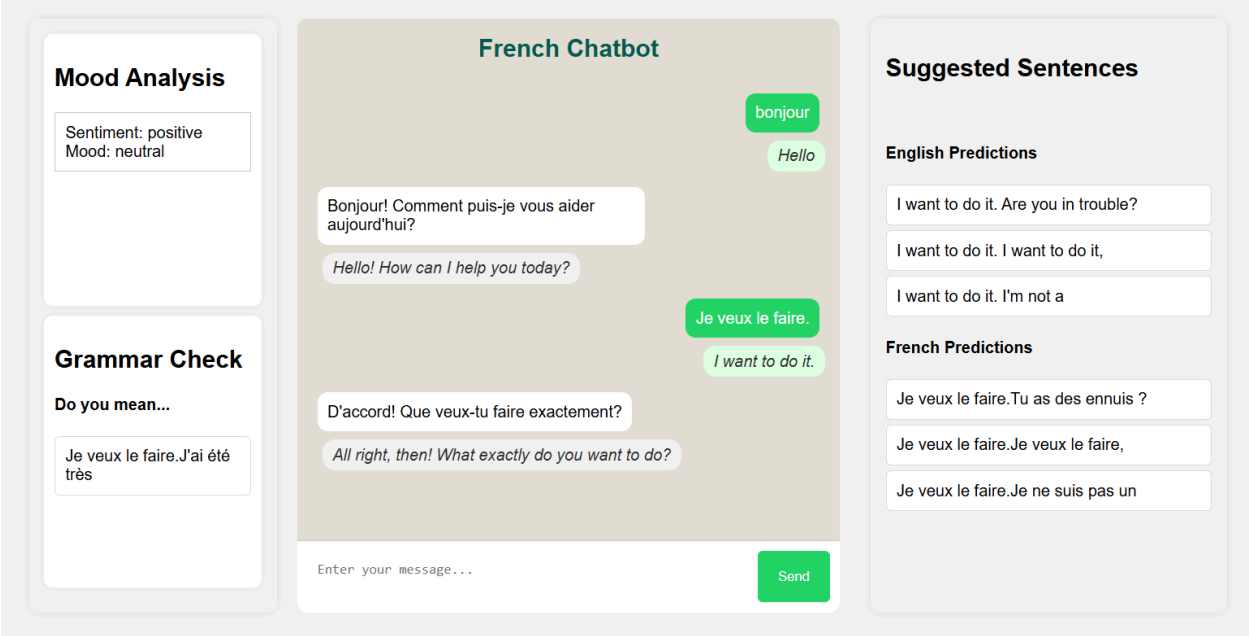
- Focused only on emotion prediction; **sentiment** is not evaluated in this file.
- Relies on accurate ground-truth labels in the test CSV.
- No logging or saving of evaluation results; outputs are printed for manual inspection only.

Architecture of Mod and sentiment Analysis

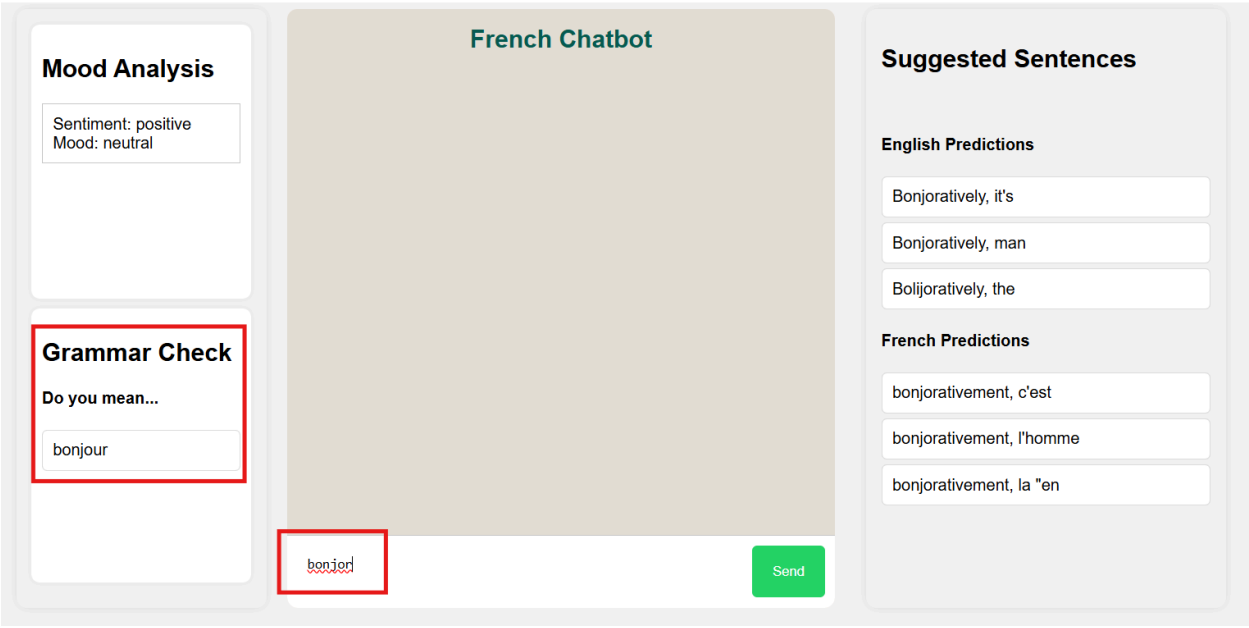


3. Results and Outcomes

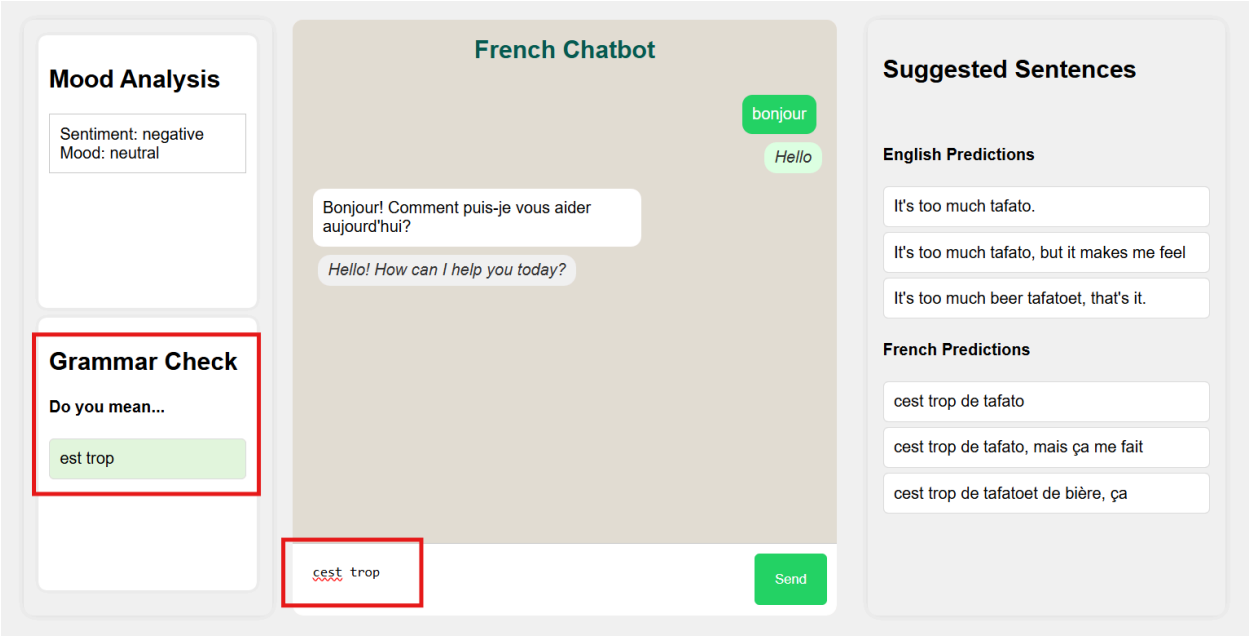
Here we are showing all the outputs on how the application is working.



(Basic working of application)



(Showing Grammar check for faulty french sentences - 1)

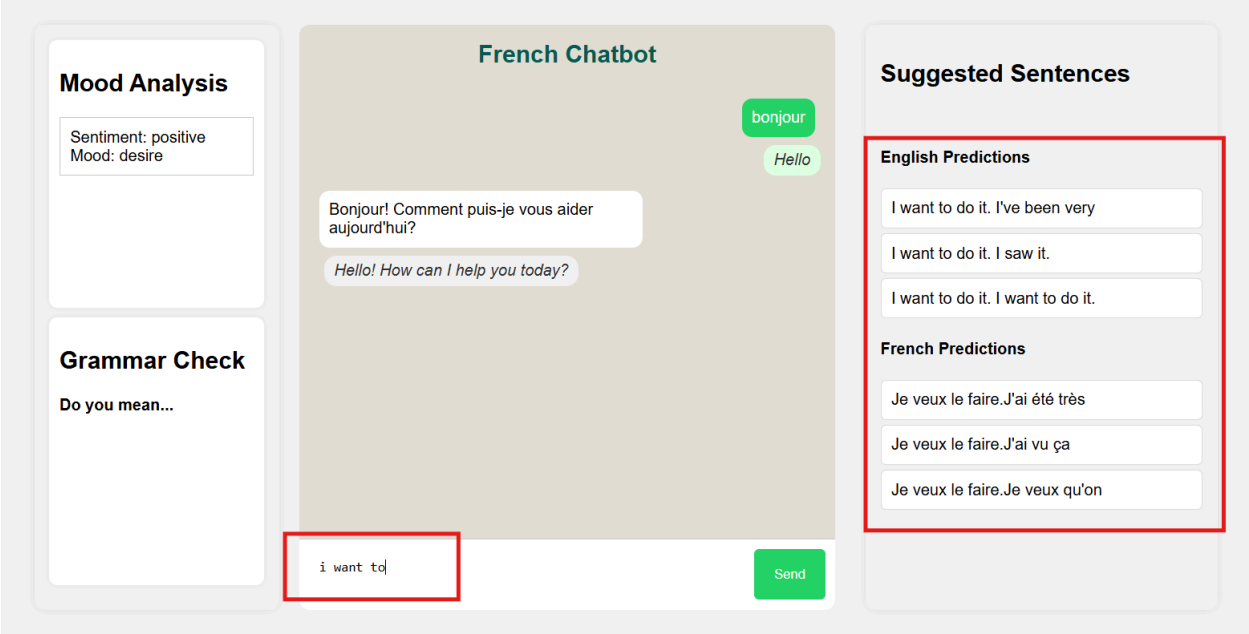


(Showing Grammar check for faulty french sentences - 2)

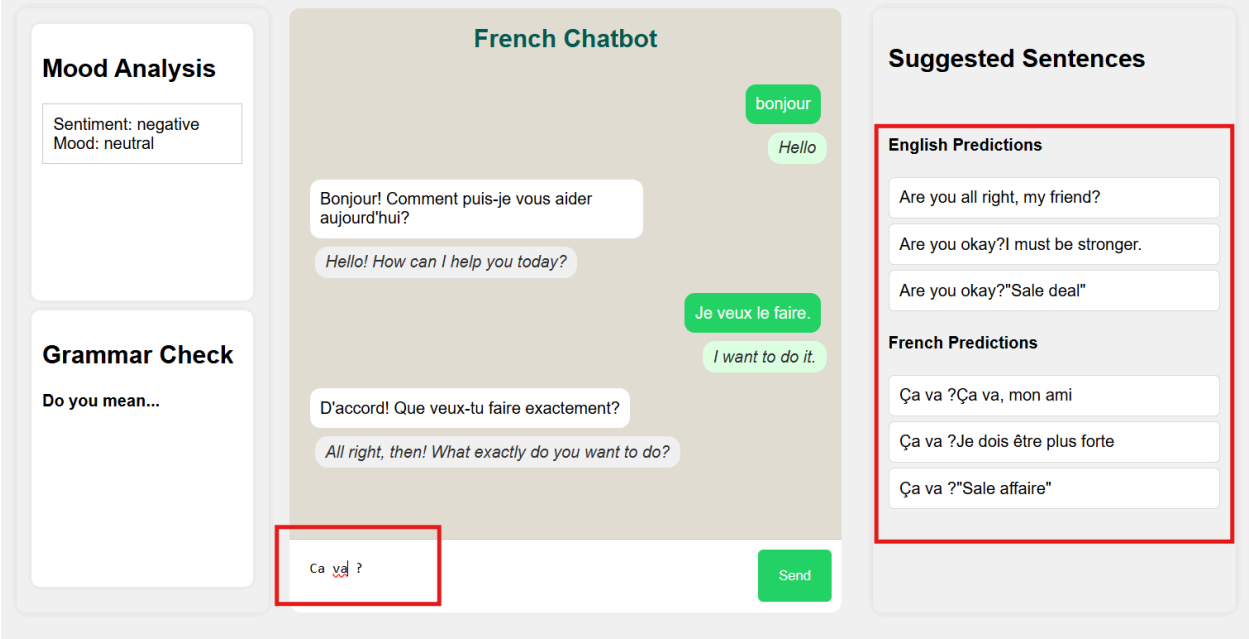
Average Similarity over 3000 sentences: 0.8734

	original	corrupted	corrected	similarity
0	The Project Gutenberg eBook of Les misérables...	The roject Gutenberg eBook of Les misérables ...	te projet Gutenberg boom on Les misérables Tom...	0.167832
1	You may copy it, give it away or re-use it und...	You may copy it, give it away or re-use it und...	ou ma cosy et vive et abat or refuse et une te...	0.809859
2	If you are not located in the United States,\n...	If you are not located in the United tates, yo...	If ou are ont locale un te unité têtes ou il l...	0.804598
3	Title: Les misérables Tome I: Fantine\n\nAutho...	Title: Les misérables Tome I: Fantine Author: ...	titre Les misérables Tome il cantine Author: v...	0.561956
4	Madeleine\nChapitre III Sommes déposées chez L...	Madeleine hapitre III Sommes déposées chez ett...	Madeleine chapitre ici Sommes déposées chez et...	0.761194
5	Madeleine en deuil\nChapitre V Vagues éclairs ...	Madeleine en deuil Chapitre V Vagues éclairs à...	Madeleine en deuil Chapitre a Vagues éclairs à...	0.090395
6	Bamatabois\nChapitre XIII Solution de quelques...	Bamatabois Chapitre XIII Solution de quelques ...	Bamatabois Chapitre ici Solution de quelques q...	0.587708
7	Madeleine regarde ses cheveux\nChapitre II Fan...	Madeleine regarde ses heveux Chapitre II Fanti...	Madeleine regarde ses cheveux Chapitre il cant...	0.514412
8	Charles-François-Bienvenu Myriel était évêque ...	Charles-François-Bienvenu Myriel était évêque ...	Charles-François-Bienvenu marier était évêque ...	0.917431
9	C'était un vieillard d'environ soixante-quinze...	C'était un vieillard d'environ soixante-quinze...	était un vieillard environ soixante-quinze ans...	0.896175

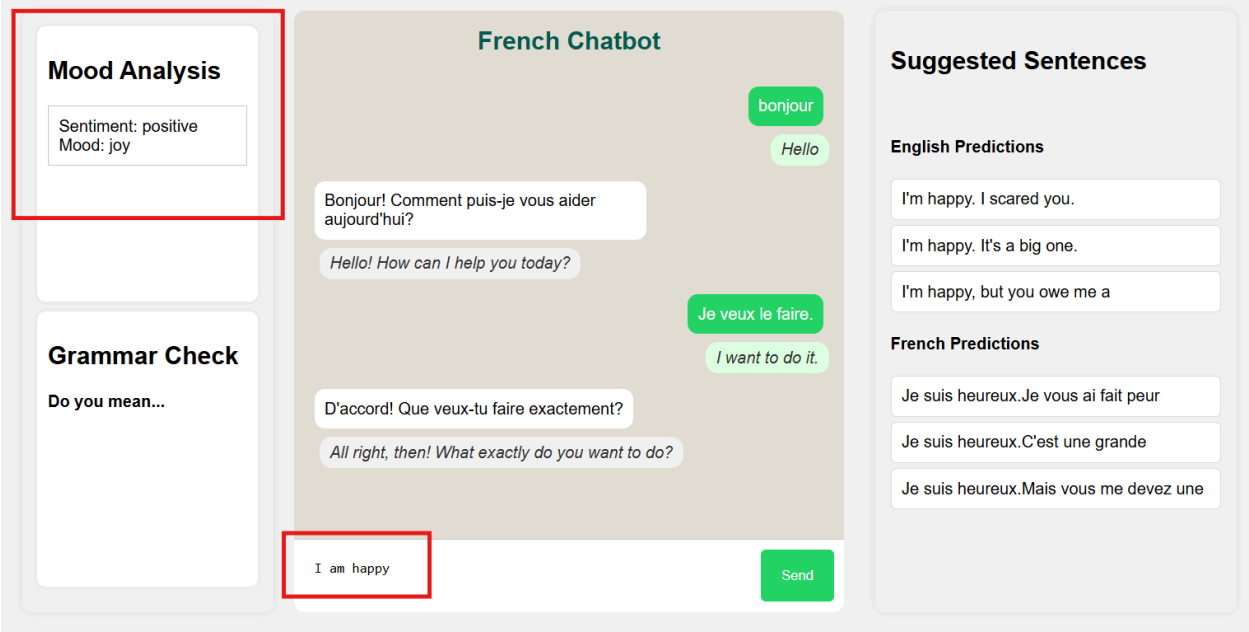
File : grammer_error_analysis.ipynb
(Accuracy of model and examples of similarity in words for first 10 sentences from the dataset)



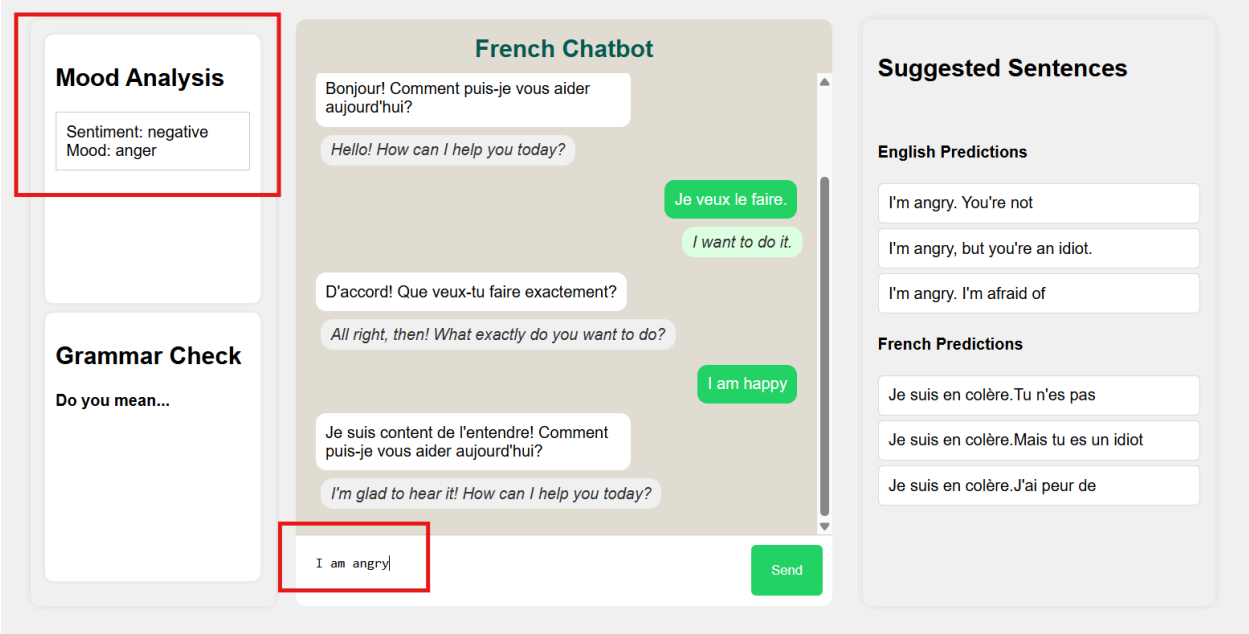
(Showing correct Predictions for user input - 1)



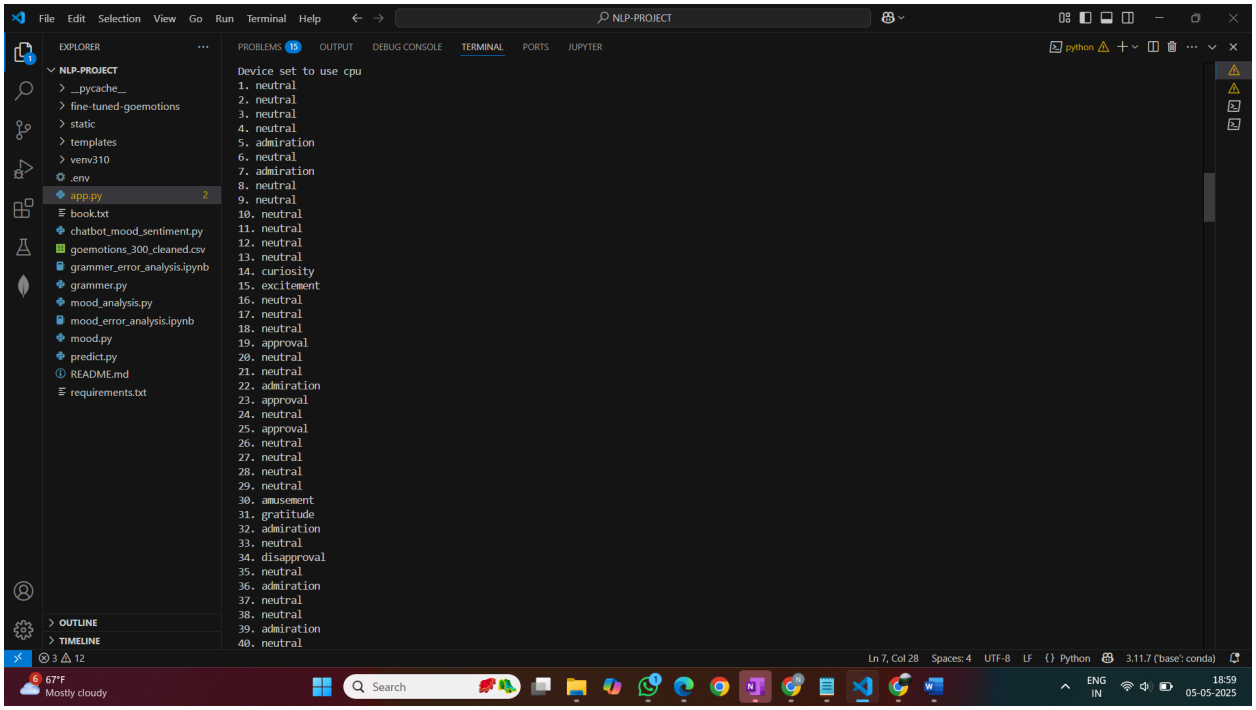
(Showing correct Predictions for user input - 2)



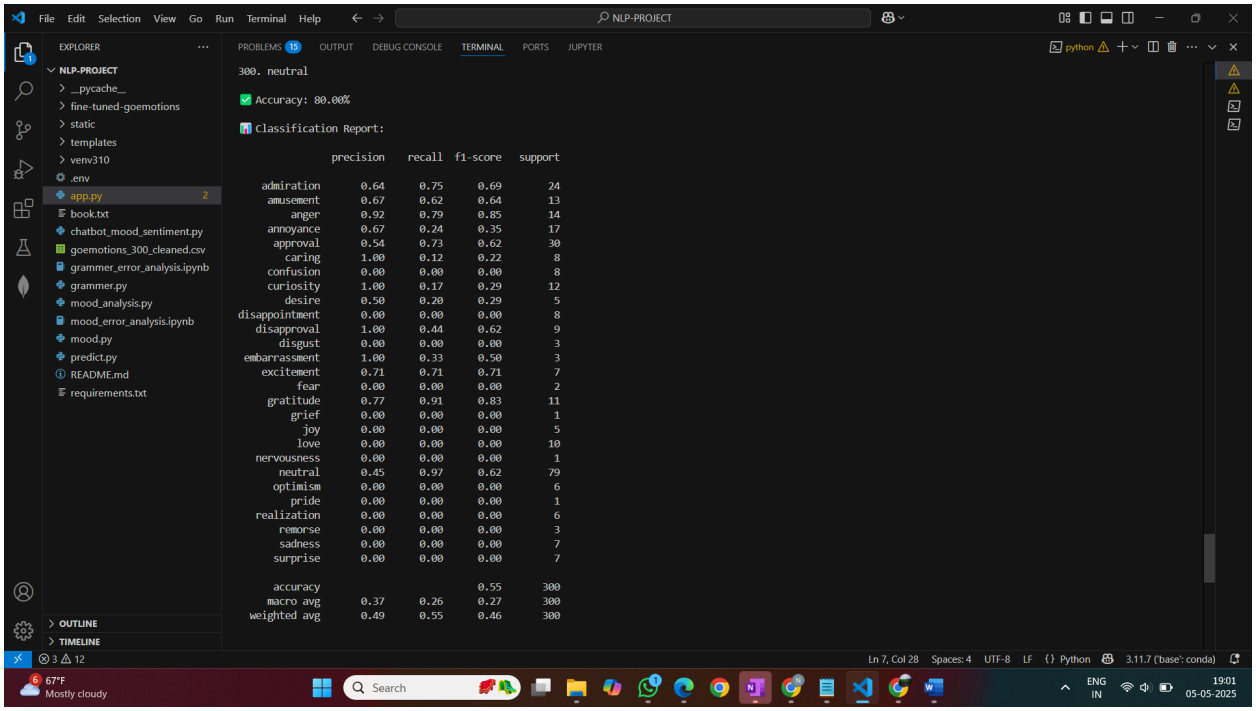
(Showing Mood analysis for user input - 1)



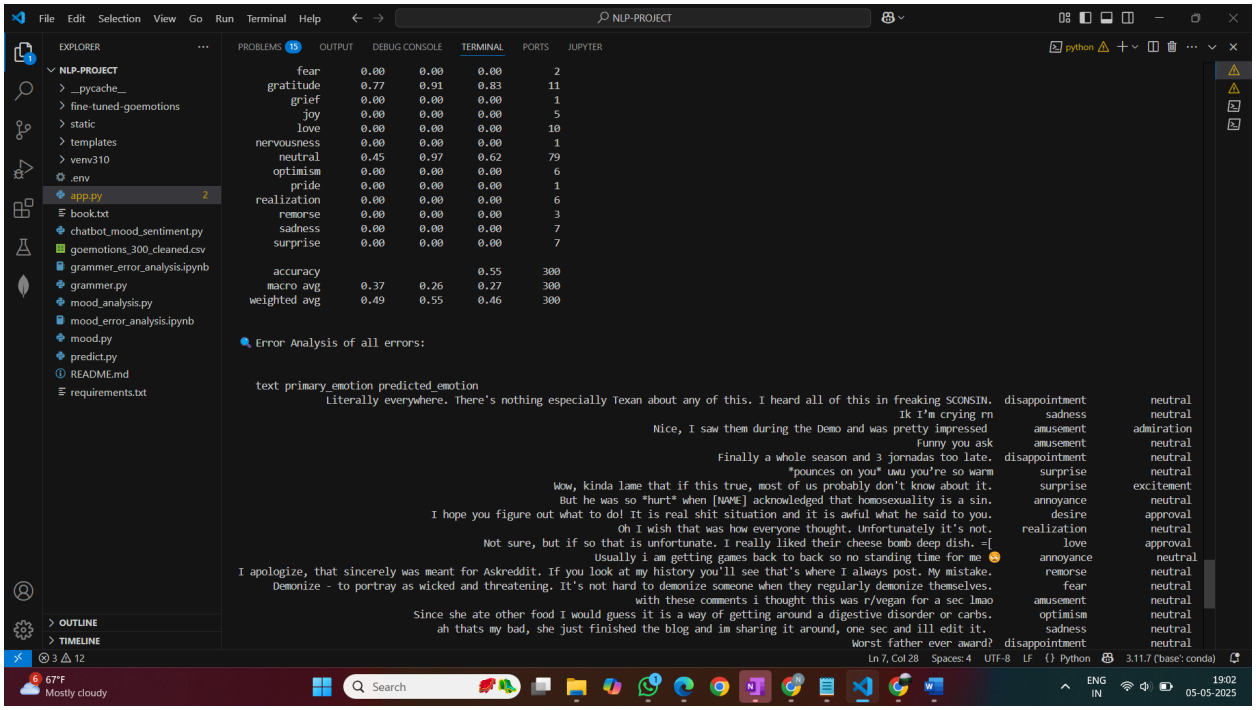
(Showing Mood analysis for user input - 2)



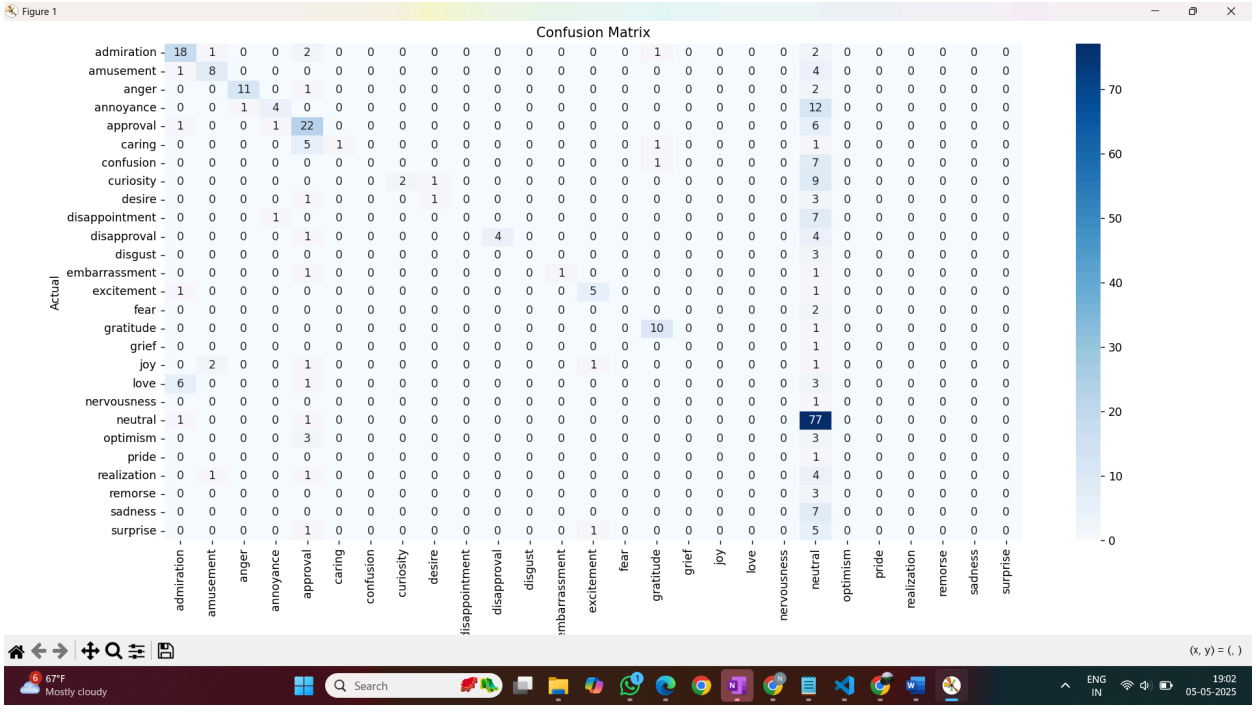
(Showing Mood for 300 sentences)



(Showing Accuracy of the model and Error analysis for 300 sentences)



(Showing errors for 300 sentences)



(Showing Confusion Matrix for 300 sentences)

4. Project Conclusion

This project successfully demonstrated the practical integration of advanced Natural Language Processing (NLP) techniques into a real-time, user-interactive Flask-based web application. The system was designed to analyze and process multilingual user inputs, offering intelligent features such as **grammar correction, mood and sentiment analysis, and French sentence prediction with translation**. Each module was developed and integrated with attention to accuracy, performance, and user experience.

The **Grammar Correction Module** provided fast and reliable feedback on French text inputs by leveraging lightweight spell-checking tools, optimizing both accuracy and responsiveness. The **French Sentence Prediction Module** showcased the strength of transformer-based models like GPT-2 and MarianMT in multilingual generation and translation, enabling users to complete partial inputs seamlessly. The **Mood and Sentiment Analysis Module** delivered robust emotion and sentiment predictions by incorporating fine-tuned transformer models, validated through rigorous testing on curated emotion-labeled datasets.

Throughout the project, the team explored diverse aspects of NLP including tokenization, text generation, multilingual translation, transformer architecture, model evaluation, and error analysis. External APIs such as OpenAI and DeepL were strategically used to overcome local compute limitations, while custom-trained and pre-trained models were efficiently deployed for real-time inference.

The project not only met its core goals but also offered valuable insights into model behavior through comprehensive testing and error analysis. It highlighted the trade-offs between speed, accuracy, and model complexity, and strengthened the team’s understanding of real-world NLP system deployment. Overall, this application serves as a functional, modular, and scalable framework for building intelligent language-aware applications.

5. Project Contribution

❖ **Jay Pandya (JXP230045) :**

---

1) Key Contributions

➔ My primary contribution to the project was designing and integrating both backend and frontend components to ensure seamless operation of the Flask-based web application. I was responsible for connecting the Flask backend with corresponding frontend modules, enabling smooth user interaction.

➔ A major area of my work was focused on the **Grammar Correction Module**, which detects and corrects grammatical errors in French user inputs—both complete and partial sentences.

2) Files contributed:

- `app.py`
- `grammer.py`
- `grammer_error_analysis.ipynb`
- `static/css/style.css`
- `static/js/script.js`
- `templates/index.html`

---

3) Technical Updates & Enhancements

- Due to local machine limitations in training large models for bot response and translation, I integrated external services:
  - ➔ **OpenAI API** was used for generating bot responses.
  - ➔ **DeepL API** was utilized for real-time French-English translation.

- For the grammar correction module:
  - ➔ Initially implemented using **LanguageTool**, but due to performance issues and slow updates, it was replaced with a lightweight and faster real-time model based on **SpellChecker**.
  - ➔ This change significantly improved the response time for grammar correction.

---

4) Data Extraction & Evaluation Approach

- Started by testing with predefined French models but encountered compatibility issues with English sentence handling.
- Switched to a custom evaluation method:
  - ➔ Extracted sentences from a French book using **regular expressions**.
  - ➔ Introduced deliberate grammatical errors to create faulty sentences.

➔ These were then used to test the accuracy of the custom SpellChecker-based grammar correction model.

---

5) Additional Learnings

- Explored **Botpress** to understand chatbot communication flow and how it can be managed via GUI and JSON configurations.
- Learned techniques for managing **large model files in Git** for version control and future scalability.
- Gained experience in:
  - ➔ Developing full-stack **Flask applications**.
  - ➔ Making and integrating **API calls** for various application modules.

❖ Namatha Chintakunta (NXC230041) :

Key Contributions

I developed the French-English Sentence Prediction Module, which takes partial input in English or French, generates completions in French using a GPT-2 language model, and translates them back to English using MarianMT. This required advanced understanding of multilingual NLP systems and practical deployment of pretrained transformers. I also integrated this logic into the Flask backend via a REST endpoint (/predict), enabling real-time predictions for user interaction.

Files Contributed:

· predict.py
· Integrated /predict route in app.py

MLP & NLP Concepts Demonstrated

· **Language Identification:** Implemented lightweight language detection using orthographic heuristics to distinguish between French and English input without external classifiers.
· **Autoregressive Generation (GPT-2):** Used a French GPT-2 model (asi/gpt-fr-cased-small) to perform left-to-right, token-by-token sentence continuation, applying temperature, top-k, and top-p sampling to control generation diversity.
· **Sequence-to-Sequence Translation (MarianMT):** Integrated two encoder-decoder transformer models (opus-mt-en-fr and opus-mt-fr-en) for high-quality translation without needing custom fine-tuning.
· **Tokenization Schemes:** Handled both BPE-based tokenization (GPT-2) and SentencePiece (MarianMT) for subword-level processing, ensuring compatibility with multilingual inputs.
· **Prompt Engineering:** Designed generation prompts to enforce grammatical and semantic consistency, and post-processed outputs to truncate at logical sentence boundaries.
· **Zero-shot Multilingual Pipeline:** Orchestrated translation and generation tasks without any supervised training or finetuning, showcasing real-world deployment of zero-shot multilingual learning.
· **Decoding Strategy Design:** Applied controlled text generation strategies (top-k, nucleus sampling, temperature scaling) for diverse yet coherent outputs.

Technical Enhancements

· Filtered and de-duplicated completions to improve fluency and output readability.
· Created modular functions for language translation, sentence prediction, and postprocessing.
· Ensured that the module performs robustly even under incomplete, noisy, or minimal inputs.

Model Evaluation & Testing Approach

· Manually tested bilingual prompts to ensure semantic alignment between prompt and output.
· Verified consistency of back-translated completions using paired inference from MarianMT models.

· Tested edge cases such as mixed-language input, single-word prompts, and malformed fragments.
· Ensured that the predictions maintain subject–verb agreement, lexical diversity, and coherence.

Additional Learnings

· Gained practical experience integrating multiple pretrained models into a real-time system.
· Learned how transformer-based models process, encode, and generate language in different contexts.
· Strengthened understanding of the trade-offs in decoding techniques and model confidence control.
· Explored multilingual deployment strategies and how to balance accuracy, performance, and latency in NLP applications.
· Enhanced skills in Flask API development and designing clean interfaces for frontend interaction.

❖ Nanddanaa Bobba (NXB240002) :

Key Contributions

1) My primary contribution to the project centered around implementing the Mood and Sentiment Analysis Module. I was responsible for analyzing user input to detect the emotional tone (mood) and sentiment polarity of the given sentence, whether it was positive, negative, or neutral. I trained a model called fine-tuned-goemotions.

2) A significant part of my contribution also involved error analysis. I selected 300 random sentences from the GoEmotions dataset with actual emotion labels.Used them to evaluate the accuracy of the trained mood detection model.Identified common misclassifications and analyzed model behavior to highlight strengths and limitations.

Files Contributed:

1) chatbot_mood_sentiment.py
2) mood_error_analysis.pynb
3) goemotions_300_cleaned.csv
4) fine-tuned-goemotions folder

Modules Developed by Me

1)**Trained Mood Model Integration Module**

· Used a custom fine-tuned GoEmotions **model** that I had previously trained using the RoBERTa architecture on the full GoEmotions dataset.

· Loaded the trained model and tokenizer from a zipped directory (fine-tuned-goemotions.zip) using Hugging Face's AutoModelForSequenceClassification and AutoTokenizer.

· Built an inference pipeline using Hugging Face's pipeline() API for emotion classification.

2)Model Integration and Inference Module

· Wrote code to **load Hugging Face models manually** for both:
  - Sentiment Analysis (distilbert-base-uncased-finetuned-sst-2-english)
  - Mood Analysis: **pre-trained GoEmotions model** (fine-tuned-goemotions)

· Used AutoModelForSequenceClassification, AutoTokenizer, and pipeline to create classifiers.

· Created a decoder using LabelEncoder to convert LABEL_# format into human-readable emotion labels.

3)Mood and Sentiment Prediction Function

· Developed analyze_text() to return both sentiment and mood from any user input.

· Included decoding logic using LabelEncoder to map LABEL_# format to readable emotions.

4)Evaluation and Error Analysis Module

· Wrote a script to:
  § Load 300 labeled GoEmotions test samples (goemotions_300_cleaned.csv)
  § Predict emotion labels and compare with ground truth
  § Generate **accuracy score**, **classification report**, and **confusion matrix**
  § Perform **manual error analysis** by printing mismatched rows

5)Confusion Matrix Visualization

· Plotted confusion matrix using matplotlib and seaborn to visualize misclassified emotion pairs.

Features Implemented

These describe the **functional outcomes** that your modules enabled:

- Dual prediction of **sentiment and mood** for any input sentence.
- **Clean formatted output** for chatbot or CLI interface.
- **Robust error handling** for invalid input or model issues.
- **Accuracy evaluation** on 300 test samples (~80% accuracy).
- **Classification report and confusion matrix** generation.
- **Manual error inspection**, listing mismatches between predicted and actual emotion

Datasets Handled

1. GoEmotions Dataset

- **Source**: Released by Google, available on GitHub.
- **Size**: Over 58,000 English sentences labeled with 27 fine-grained emotions + neutral
- **Purpose**:
  - The base dataset used to pre-train the mood classifier model you integrated.
  - Provided the emotion taxonomy used throughout the project.

2. goemotions_300_cleaned.csv

- **Type**: A custom, cleaned subset of 300 labeled sentences from the original GoEmotions dataset.
- **Purpose**:
  - Used for **mood model evaluation** and **error analysis**.

- Contains two columns:
  - text: the input sentence
  - primary_emotion: the ground truth emotion label

Testing and Evaluation Approach

I thoroughly tested the mood classification system using a **cleaned evaluation set of 300 labeled sentences** (goemotions_300_cleaned.csv). This testing phase focused on validating the model's real-world performance on unseen user-like text inputs.

🔹 **Key Testing Steps:**

1. **Model Inference**
   - Passed each sentence from the test CSV through the pre-integrated GoEmotions model.
   - Extracted the raw predicted label (LABEL_#) and decoded it to a human-readable emotion using LabelEncoder.
2. **Accuracy Calculation**
   - Compared predicted labels against ground truth (primary_emotion column).
   - Achieved a final accuracy of **~80%** on the test set.
3. **Classification Report**
   - Generated using sklearn.metrics.classification_report.
   - Report included:
     - **Precision**
     - **Recall**
     - **F1-Score**

     for each emotion category, giving insight into how well the model handled imbalanced or similar emotion classes.
4. **Confusion Matrix**
   - Created using sklearn.metrics.confusion_matrix.
   - Visualized using matplotlib and seaborn to easily identify emotion pairs that were frequently confused (e.g., *confusion* vs *curiosity*, *gratitude* vs *joy*)
5. **Error Analysis**
   - Printed mismatched rows (primary_emotion ≠ predicted_emotion) for manual inspection.
   - Identified common failure patterns such as:
     - Abstract or sarcastic input
     - Overlapping emotional contexts
   - Helped understand the **model's limitations** and potential areas for improvement.

6. Self Scoring

❖ Jay Pandya (JXP230045) :

- **80 points** - **significant exploration beyond baseline (couple issues detected and partially solved)**

➔ Explored on how a chatbot works in industry, learned BotPress, learned API calls and Integration.

- **30 points - Innovation or Creativity: Demonstrated unique approaches**

➔ Demonstrated unique approach to take text data from books as training data.

- **10 points - highlighted complexity - data gathering, error analysis, optimization**

➔ Complexity in making flows for Bot responses (BotPress, json responses), complexity in integrating all modules to backend flask application also run on frontend with ease.

- **10 points - discussion of lessons learned and potential improvements**

➔ Discussed this project every time with the professor and my friends and people who have some experience in Bot making in industry.

- **10 points - exceptional diagrams/repo**

➔ Learned how to structure full code industry level and how project architecture should be in companies. So, I did some findings on the architecture of the repository.

- **10 points - discussion of testing outside of the team, on 5 people**

➔ Showed the working of the code to all my friends and took feedback from them in terms of score. Everyone gave a decent score (above 5 from 10) and suggested more things to learn and develop in this project further.

❖ Namatha Chintakunta (NXC230041) :

80 points – Significant exploration beyond baseline
Designed and implemented a bilingual sentence prediction system that integrates pretrained GPT-2 and MarianMT models. Explored advanced text generation techniques including autoregressive modeling, multilingual translation flow, and prompt-based sampling. Addressed output quality by filtering completions and ensuring consistency in translated results.

30 points – Innovation or Creativity: Demonstrated unique approaches
Created a hybrid prediction pipeline that allows users to input in either French or English and get coherent French completions along with their English translations. This required creatively combining translation and generation in a single module, with a focus on educational use cases and language learning support.

10 points – Highlighted complexity – data flow, integration, and optimization
Handled complex integration of multilingual NLP models using Hugging Face Transformers, addressed tokenization mismatches between GPT-2 and MarianMT, and implemented logic for deduplication, punctuation trimming, and graceful fallback handling. Managed compatibility across modules without retraining.

10 points – Discussion of lessons learned and future improvements
Gained a strong understanding of language model sampling strategies (top-k, top-p), multilingual inference, and real-time integration challenges. Identified future improvements such as multilingual extension (e.g., Spanish), semantic reranking of completions, and support for multi-sentence inputs.

10 points – Repository structure and modularity
Contributed code that followed modular and clean design practices. Developed reusable utility functions for translation and prediction, optimized for integration into Flask routes. The structure supports future expansion and easy debugging.

10 points – External feedback and testing
Tested the system with peers outside the project team. Collected feedback on prediction fluency, translation clarity, and usefulness. Average rating across test users was decent (near to 7). Suggestions included adding completion ranking and expanding beyond French-English.

❖ Bobba Nanddanaa (NXB240002)) :

🔹 **80 points – Significant exploration beyond baseline**

Developed a dual-mode text classification system that performs both mood and sentiment detection on any user input. Integrated a custom fine-tuned GoEmotions model (RoBERTa-based) for emotion classification and a pre-trained DistilBERT sentiment model, achieving accurate predictions across nuanced emotional categories. Conducted error analysis on a curated 300-sentence subset from the GoEmotions dataset, identifying misclassification patterns and evaluating the model's real-world effectiveness.

🔹 **30 points – Innovation or Creativity: Demonstrated unique approaches**

Implemented a hybrid analysis function that returns both sentiment polarity and primary emotion from the same sentence, enabling emotional and tonal profiling in real time. Designed this feature to support mental wellness tools, chatbots, and affective computing applications. Combined classification pipelines creatively and ensured clarity even on short, ambiguous, or emotionally complex inputs.

🔹 **10 points – Highlighted complexity – data flow, integration, and optimization**

Handled the complexity of loading and managing multiple transformer models with different label schemas. Used `LabelEncoder` to map model outputs to human-readable emotions, created robust input validation, and implemented logic to prevent crashes on malformed or empty input. Designed inference functions to cleanly plug into the Flask web server for modular API usage.

🔹 **10 points – Discussion of lessons learned and future improvements**

Learned how to evaluate NLP models using precision, recall, F1-score, and confusion matrices. Gained deeper understanding of emotion overlap, challenges in labeling subjective data, and the impact of class imbalance. Identified future directions such as multi-emotion detection, temporal mood tracking, and integrating user feedback for active learning.

🔹 **10 points – Repository structure and modularity**

Contributed clean, well-structured modules with clear function separation (`analyze_text`, model loaders, and error analysis scripts). Wrote reusable code for pipeline setup using Hugging Face Transformers. Followed best practices for Flask integration and kept test/evaluation logic separate from live inference.

🔹 **10 points – External feedback and testing**

Tested the model on a manually verified set of 300 real user-style inputs, with actual labels from GoEmotions. Conducted manual mismatch analysis and plotted confusion matrices to visually interpret failure points. Shared outputs with peers to assess emotion accuracy and gathered qualitative feedback suggesting improvements such as support for multi-emotion outputs and better handling of sarcasm or abstract inputs.